
Why your plan needs a **smarter reviewer**

为什么写计划的模型，需要另一个（通常更强的）模型来 review ——
以及具体怎么做。

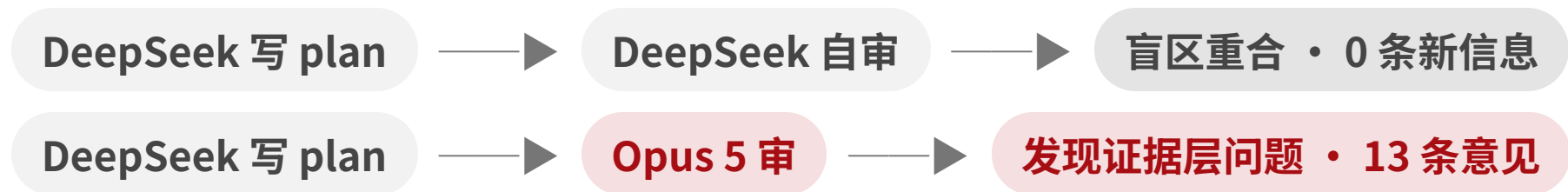
What's in here

- 01 **Why** 同一个模型，同一片盲区
- 02 **The receipt** 唯一错的小节，恰是唯一没测的
- 03 **Why stronger** 两个机制，不只是「更聪明」
- 04 **Where & How** 评审花在不可逆处；分屏、收敛、再写码
- 05 **Cheat sheet** 三条守则

Same model, same blind spot

- 生成是展开，评审是证伪 —— 两种不同的能力。
- 自评和写作走同一条证据链：错的证据不会触发警报。

结论错了能改；证据错了，会一路带进代码。



The only wrong section was the **only untested one**

01 证据密度和两处自我推翻值得保留。

02 一句 JSDoc 交叉引用被当成了「声明证据」。

03 唯独纯读文档那节，错了。

那条是把 JSDoc 里的一句错误交叉引用当成了声明证据。其余“实跑验证”的条目（marked 12 载荷、事件顺序）我抽查都站得住，唯独 webview 宿主 API 那节是纯读文档得来的，也正是唯一出错的一节。建议 §0.5

Two mechanisms, not one

为什么是「更高智力」，而不只是「另一个模型」。

A 能力不对称

只有更强的模型能分辨「看起来有证据」和「真的~~有~~证据」。

B 失效模式不同

不同训练数据 → 幻觉与疏漏的分布不一样。交叉评审覆盖的是盲区。

执行模型

↻ 66轮 511步 ⓘ 109M tok · 缓存命... deepseek-flash(max)

便宜、快

评审模型

* Cogitated for 5m 1s · done 18:52

贵、慢 · 只在刀口上

Review the irreversible

- 不可逆、有安全边界、有下游依赖的决策 —— 必须外部评审。
- 反例：不要 review 每一行代码。

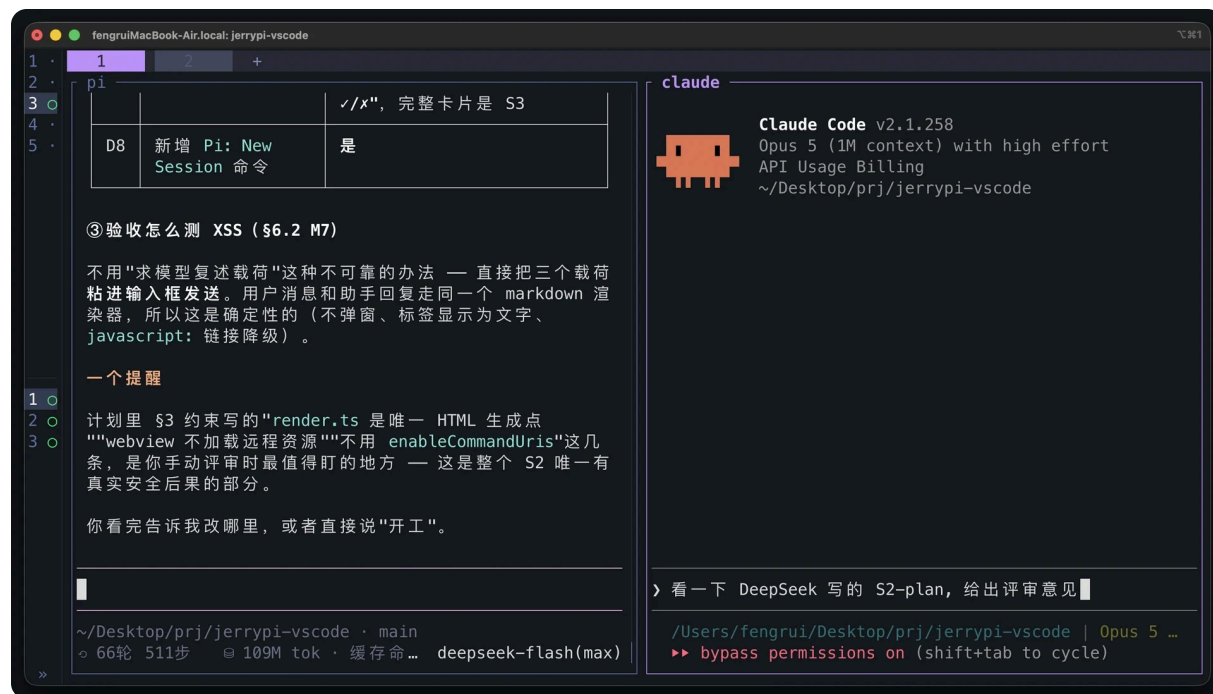
这条错了要返工，就 review 它。

一个提醒

计划里 §3 约束写的 `"render.ts` 是唯一 HTML 生成点
`""webview 不加载远程资源""`不用 `enableCommandUris` 这几条，是你手动评审时最值得盯的地方 — 这是整个 S2 唯一有真实安全后果的部分。

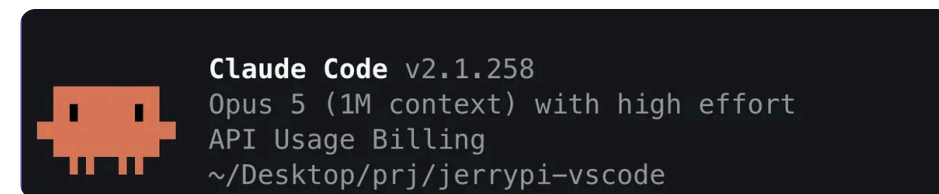
屏幕里的原话：「**这是整个 S2 唯一有真实安全后果的部分。**」

Split the screen, hire a reviewer



左：DeepSeek（执行 / 写 plan） · 右：Opus 5（评审）

评审员和你同目录，能自己读 plan、读代码、跑测试——不用你贴上下文。



选型：1M context 装得下整个 repo；high effort 值这几分钟。

Converge, then code

按我们的流程，我先写 `docs/S2-plan.md`，然后跑外部评审（Claude + Codex），收敛后再动手。

要我现在开始写 S2 计划吗？

一句话启动

- 01 把流程**写进约定**（skill / AGENTS.md）—— AI 自己会拉评审，不用你提醒。
- 02 一句话启动： 看一下 DeepSeek 写的 S2-plan，给出评审意见
- 03 意见**编号落盘**： S2-CL-01... → plan 的 § 10 评审记录。
- 04 喂回执行模型： 阅读 herdr 右侧分屏中 opus 的评审意见。
- 05 收敛后才动手 —— **拍板的是人**。

Three rules

01 别让写的人自己审。

自评看不见证据层的错 —— 那条错误对它来说也是「已确认的事实」。

02 评审员要更强、要不同，只花在不可逆的 5% 上。

便宜模型跑量，贵模型把关。别平均用力。

03 意见必须编号落盘。人是拍板的人。

S2-CL-01 → § 10 评审记录。可追溯，才收敛得下来。